

SID – Práctica 3 (Laboratorio)

Mayo/Junio 2026

1. Introducción

En las sesiones de laboratorio, en la parte de aprendizaje por refuerzo multi-agente, vemos diversos algoritmos considerados *fundacionales* debido a que son la base de métodos más modernos (y más complejos). El primero que hemos visto es Joint Action Learning with Game Theory (JAL-GT), que consiste en un Q-Learning donde el estado puede ser una observación parcial (o un histórico de observaciones, aunque por simplificar no vamos a ir por esta vía) y la acción es una acción conjunta. Los valores Q se utilizan para construir conceptualmente una matriz de recompensas, sobre la que se pueden aplicar soluciones de concepto como Minimax, eficiencia de Pareto o equilibrio de Nash.

2. Objetivo de la práctica

A partir de un código que os pasamos¹ y que define un *baseline* o modelo de referencia, se os pide que realicéis un estudio del rendimiento del algoritmo JAL-GT sobre el entorno POGEMA². El foco de esta práctica está en formular hipótesis, ejecutar comparativas bien justificadas e interpretar los resultados en base a los conceptos vistos en teoría y laboratorio.

2.1. El entorno: POGEMA

POGEMA (*Partially-Observable Grid Environment for Multiple Agents*) es un entorno bidimensional multiagente con observabilidad parcial. El entorno está representado por una cuadrícula con una anchura y una altura determinadas, con la presencia de obstáculos que los agentes no pueden cruzar ni ocupar. Como mínimo, el perímetro del mapa está formado por obstáculos (son los muros que delimitan el mapa). Además, puede haber posiciones aleatorias en la cuadrícula que también son obstáculos y, de existir, vienen determinados por la semilla a partir de la cual se crea el entorno.

¹<https://drive.google.com/file/d/1A9T90bnJpIAEl0tQBCcqZypZ7Me00lmu/view?usp=sharing>

²<https://github.com/AIRI-Institute/pogema>

Los agentes comienzan en posiciones aleatorias y tienen un objetivo a alcanzar. Estas posiciones también están determinadas a partir de la misma semilla. Por defecto, sólo se entrega una recompensa con valor 1 cuando un agente llega a su objetivo correspondiente, tras lo cual ese agente desaparece. Por lo tanto, si hay dos agentes y los dos cumplen su objetivo, la recompensa total del entorno será 2. No hay recompensa ni un agente desaparece si accede a una posición en la que hay un objetivo de otro agente.

La ejecución del entorno es, por defecto, paralela y no por turnos. Es decir, todos los agentes reciben sus observaciones y deciden sus acciones a la vez. Dos agentes no pueden ocupar la misma posición en el mismo paso de ejecución. Si dos agentes pueden acceder a la misma posición y lo intentan en el mismo paso, el agente con identificador más pequeño accede a la posición y el otro no se mueve (*hay un “choque”*).

La observabilidad es parcial y viene determinada por un parámetro que se pasa al entorno en su creación: el radio de observación. Todos los agentes tienen el mismo radio de observación. Cada turno, cada agente recibe observaciones correspondientes a una cuadrícula $(x - r, y - r, x + r, y + r)$, donde (x, y) es la posición del agente después de la última acción y r es el radio de observación. Es decir, si $r = 1$ entonces la cuadrícula es de 3×3 , si $r = 2$ la cuadrícula es de 5×5 y así sucesivamente. La coordenada del agente siempre está en el centro de la cuadrícula.

La observación que recibe cada agente consta de tres matrices (*Numpy arrays*):

- Matriz de obstáculos con un valor de 1 si hay obstáculo en esa posición, 0 si no lo hay.
- Matriz de agentes con un valor de 1 si hay un agente en esa posición (incluyendo el agente que recibe la observación), 0 si no lo hay.
- Matriz de objetivo con un único valor de 1 en la posición de la cuadrícula más cercana al objetivo de ese agente. No aparecen objetivos de otros agentes y siempre hay un 1 (y sólo uno).

Para poder generar políticas o funciones de valor a partir de estas observaciones, es necesario convertirlas en estados discretos (si no utilizamos modelos más complejos). Para ello, hemos incluido una función `obs_to_state` con un ejemplo de conversión. En teoría, deberíamos estar usando históricos de observaciones como estados, ya que la observabilidad es parcial, pero en el entorno POGEMA esta simplificación es válida y hace que podamos plantear una experimentación viable con los modelos que usamos basados en *Numpy arrays*.

Por defecto, todas las ejecuciones de evaluación generan animaciones en formato SVG (se pueden abrir en un navegador, o en vista previa en algunos sistemas operativos) donde se puede observar el comportamiento de cada agente. Interpretar cualquier política se hace muy complicado con el tamaño del espacio de estados así que es muy recomendable utilizar estas animaciones para el análisis y vuestra documentación.

Antes de empezar a experimentar, dedicad un tiempo a mirar todos los parámetros de configuración de la estructura `exp_config` y entender bien en qué afecta cada uno.

2.2. Qué incluye el código *baseline*

En el código que os pasamos se incluyen los siguientes elementos:

- Una versión del algoritmo JAL-GT parecida a la del notebook de la sesión de laboratorio, con algunas optimizaciones y mejoras (`algorithms.py`).
- Una clase para almacenar información relevante del modelo de juego, incluyendo el espacio de acciones conjuntas (`game_model.py`).
- Varios conceptos de solución implementados (`solution_concepts.py`):
 - Minimax
 - Equilibrio (mixto³) de Nash
 - Bienestar (*welfare*)
 - Óptimo de Pareto⁴
- Un fichero de funciones auxiliares que sólo incluye la función para dibujar gráficas con Seaborn que hemos usado durante el curso (`utils.py`).
- El código para convertir observaciones en estados (`obs_to_state`), configurar el entorno (estructura `exp_config`), y entrenar y evaluar, en dos versiones: un fichero ejecutable (`main.py`) y un notebook (`baseline.ipynb`).
- Un fichero de dependencias (`requirements.txt`), que podéis instalar desde vuestro entorno virtual con el comando `pip install -r requirements.txt`.

Podéis modificar estos ficheros tanto como lo necesitéis para vuestra experimentación y análisis.

2.3. Experimentos obligatorios

La práctica debe incluir los dos bloques de experimentación siguientes:

- **Comparativa algorítmica.** Comparad JAL-GT frente a IQL (*independent Q-Learning*)⁵. La comparativa debe analizar, como mínimo, la recompensa

³No es un equilibrio mixto calculado de manera formal como lo hicimos en sesiones de problemas debido al coste de construir y resolver las ecuaciones lineales. Por eficiencia lo hemos implementado de la siguiente manera: si hay más de un equilibrio de Nash para un agente, se reparte la probabilidad uniformemente respecto a todos los que haya.

⁴De manera análoga a como lo hacemos para el equilibrio de Nash, si hay más de un óptimo de Pareto repartimos la probabilidad uniformemente.

⁵La mejor manera de hacer esta comparativa es entrenando a los agentes con el mismo algoritmo Q-Learning que usasteis para FrozenLake, con cada agente ignorando el hecho de que el entorno es no-estacionario debido a la presencia de otros agentes.

individual y colectiva, la estabilidad o convergencia del entrenamiento y el efecto de la no estacionariedad multiagente sobre IQL.

- **Comparativa de coordinación dentro de JAL-GT.** Comparad al menos dos conceptos de solución entre los ya incluidos en el *baseline* (minimax, Nash, *welfare*, Pareto). Debéis discutir qué tipo de comportamiento inducen, en qué situaciones parecen más adecuados y cómo cambian la coordinación y las recompensas observadas.

En ambos bloques debéis formular hipótesis previas, justificar la selección de parámetros que vais a probar y contextualizar los resultados con respecto a las propiedades de los algoritmos.

2.4. Parámetros permitidos y alcance

Para la nota base, la experimentación debe mantenerse dentro del siguiente alcance:

- Número de agentes: siempre 2.
- Tamaño del entorno: 4×4 , 6×6 y 10×10 .
- Densidad de obstáculos: dos valores, uno bajo y uno medio, elegidos y justificados por vosotros.
- Representación del estado: la función `obs_to_state` proporcionada en el *baseline*.
- Hiperparámetros a estudiar: ϵ y γ .

No es necesario, para la nota base, explorar *alpha decay*, nuevas señales de recompensa, estados alternativos, más agentes o mapas mayores. Estas variantes sólo deben abordarse si las planteáis como ampliación y las justificáis adecuadamente.

Para la comparativa con IQL, basta con una implementación tabular sencilla que utilice la misma codificación de estado y la misma configuración experimental base que JAL-GT. No se requieren mejoras adicionales sobre IQL para la nota base.

Para evaluar la calidad de los entrenamientos, se pueden usar los siguientes elementos, aunque no tiene por qué ser exactamente esta lista (podéis añadir o reemplazar elementos si lo justificáis):

- Tiempo de entrenamiento por episodio
- Número de episodios
- Tiempo de entrenamiento total
- Recompensa individual obtenida para algún agente
- Recompensa colectiva
- Optimalidad de la política resultante

3. Plazos y evaluación

La entrega tiene que incluir el código de los algoritmos, así como también un script o punto de entrada para lanzar los experimentos obligatorios. Puede incluir notebooks, siempre y cuando la documentación en Markdown de los mismos sea equivalente, en calidad y contenido, al que habríais realizado de haber hecho la documentación en un PDF aparte. Las únicas dependencias que podéis utilizar son las mismas que ya se incluyen en el fichero `requirements.txt`.

La fecha de entrega será el día **12 de junio (12/06/2026)**, y consistirá en:

- El código de experimentación y los algoritmos.
- Una documentación en .pdf (o su equivalente en notebooks) incluyendo:
 - Las hipótesis de partida y las decisiones de diseño de los experimentos obligatorios, incluyendo una justificación de los parámetros elegidos y sus rangos.
 - Un resumen de los resultados empíricos, apoyado por figuras o tablas relevantes.
 - Análisis comparativo de los algoritmos y sus parametrizaciones, contextualizando cada resultado con respecto a las propiedades de los algoritmos tal como se vieron en las sesiones de teoría y laboratorio.
 - Conclusiones y posibles recomendaciones sobre cómo entrenar agentes para entornos similares.
- Un fichero README con instrucciones concretas para parametrizar y ejecutar los experimentos obligatorios.

La nota base de la práctica se evaluará según la rúbrica descrita en la Figura 1. Esta nota tendrá un valor máximo de 10. Aparte de esta nota base, existe la posibilidad de sumar puntos extra que permitirían tener una nota mayor de 10, hasta un máximo de **2 puntos extra en total**. Para que una extensión puntúe, debe incluir una hipótesis, una comparación con alguno de los *baselines* de la práctica y un análisis de resultados correctamente contextualizado.

- **+0.5 puntos** por cada una de las siguientes ampliaciones, hasta un máximo de 1 punto en esta categoría:
 - Entrenar parejas de agentes usando conceptos de solución diferentes para ambos, y comparar su rendimiento contra parejas homogéneas.
 - Estudiar la generalización a mapas no vistos en entrenamiento. Es decir, una vez entrenados los agentes, ¿cómo rinden en mapas nuevos, individualmente y colectivamente?
 - Escalar el número de agentes. El número máximo de agentes no está acotado en principio y debería depender de cómo vaya la experimentación.

Criterio	Peso	Expectativa
Ejecución	30 %	El código funciona correctamente a partir de las instrucciones del README y permite reproducir los experimentos obligatorios de la práctica.
Implementación	20 %	La implementación de IQL, así como cualquier adaptación adicional necesaria para ejecutar la comparativa, es consistente con los objetivos de experimentación y con el funcionamiento teórico de los algoritmos. Cualquier ampliación adicional que se incluya debe estar igualmente bien implementada.
Diseño de experimentos	25 %	Los experimentos obligatorios están planteados a partir de hipótesis y están justificados en base al comportamiento teórico de los algoritmos y a las características del entorno. La selección de parámetros y comparativas es razonada y coherente con el alcance de la práctica.
Análisis de resultados	25 %	El resultado de los experimentos se analiza de manera correctamente contextualizada con respecto a las hipótesis de partida, a la no estacionariedad del entorno multiagente y a los incentivos de coordinación de cada algoritmo o concepto de solución. Se concluye con posibles recomendaciones de cómo entrenar agentes para entornos similares. Para el análisis se utilizan de manera correcta los outputs de los algoritmos, como por ejemplo recompensas, errores de diferencia temporal, el comportamiento de los agentes en las animaciones, etc.

Figura 1: Rúbrica de evaluación de la práctica

- Escalar el tamaño del entorno, por encima de 10×10 . El tamaño máximo no está acotado en principio y debería depender de cómo vaya la experimentación.
- Probar señales de recompensa alternativas y compararlas con la versión por defecto.
- **+1 punto** por cada una de las siguientes ampliaciones:
 - Utilizar una representación de estado alternativa a `obs.to_state` y compararla con la representación base.
 - Incorporar una mejora metodológica o algorítmica sencilla pero bien justificada (por ejemplo, una estrategia de *decay* distinta o un protocolo de evaluación más robusto) y compararla con la versión base.
 - Añadir, a la comparativa, o bien una versión de JAL-GT que use como representación del estado un modelo lineal (con actualización por

aproximación lineal) o una red neuronal, o bien un algoritmo de gradiente de política⁶.

El máximo acumulable por estas extensiones es, en cualquier caso, de 2 puntos.

La nota de esta práctica contará 1/3 de la nota global de laboratorio. Si la nota es superior al 10, para hacer media no se reducirá a 10 sino que se mantendrá superior.

⁶Podéis proponer algoritmos que no hayamos visto, o podéis escoger alguno de los que vemos en las sesiones de laboratorio: WoLF, DQN, A2C.